

Estructura de datos

Pilas

Dr. Rodrigo Vázquez López

Universidad Autónoma de la Ciudad de México
Plantel San Lorenzo Tezonco

Semestre 2026-II

UACM

Universidad Autónoma
de la Ciudad de México

NADA HUMANO ME ES AJENO

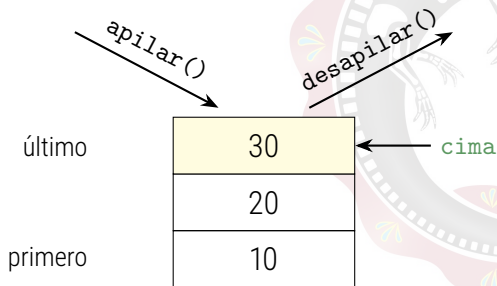
- 1 TDA pila
- 2 Implementación Pila enlazada
- 3 Apilar
- 4 Desapilar
- 5 Cima
- 6 Pila vacía
- 7 Limpiar



Tipo de dato abstracto Pila

Definición

Una **pila** es una **estructura de datos lineal** en la que las **inserciones y eliminaciones** se realizan **únicamente** en uno de sus extremos, denominado **cima**.



Principio LIFO (Last In, First Out)

El **último elemento** en **entrar** es el **primero en salir**.

Operaciones del TDA Pila

Las **operaciones fundamentales** que definiremos son:

apilar(elemento) **Agrega un elemento** en la cima.

desapilar() **Elimina y devuelve** el elemento de la cima.

cima() **Consulta** el elemento de la cima **sin eliminarlo**.

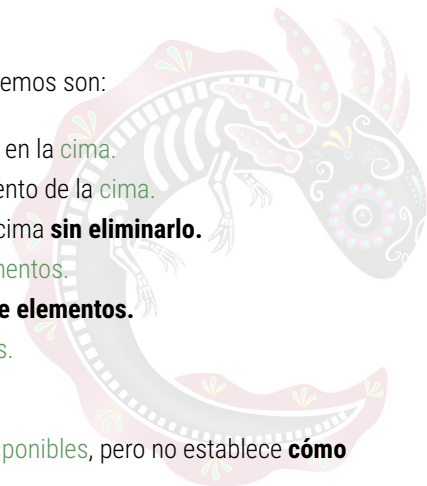
tamano() **Devuelve** el número de elementos.

estaVacia() Determina si la pila **contiene elementos**.

limpiar() **Elimina todos** los elementos.

Abstracción

El TDA especifica **qué operaciones están disponibles**, pero no establece **cómo deben almacenarse** los elementos.



Interfaz Pila<E>

```
public interface Pila<E> {  
  
    void apilar(E elemento);  
    E desapilar();  
    E cima();  
    int tamano();  
    boolean estaVacía();  
    void limpiar();  
}
```

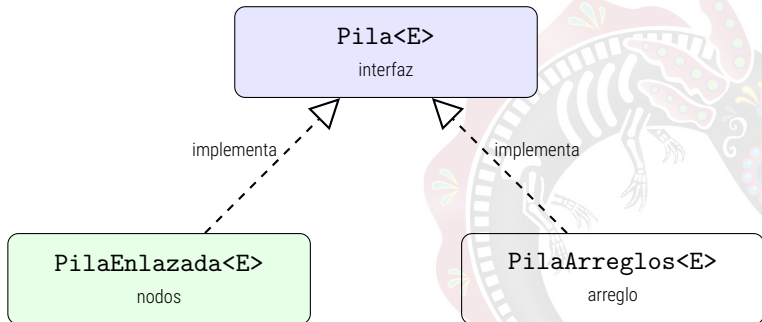
Contrato

La **interfaz** establece el **comportamiento esperado** de una pila **independientemente** de su **implementación**.

Pila<E> $\neq$ forma de almacenamiento



Interfaz e implementación

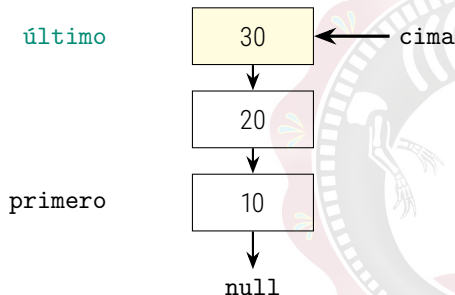


¡Importante!

Podemos cambiar la **representación interna** sin modificar las operaciones definidas por el **TDA**.

Pila enlazada

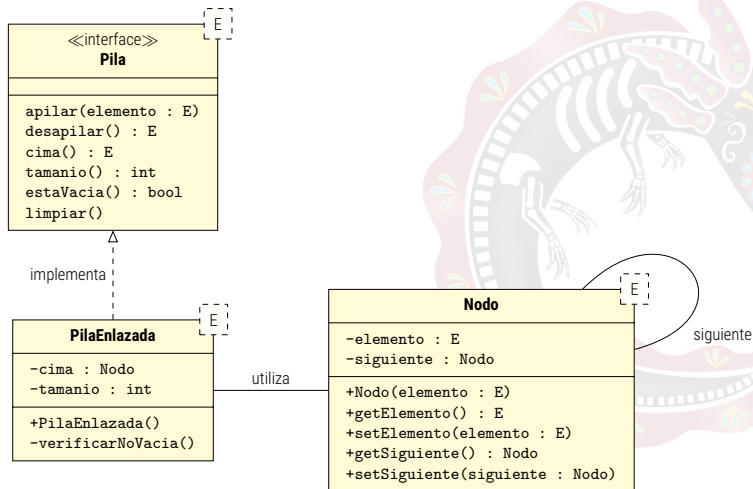
`PilaEnlazada<E>` utiliza objetos `Nodo<E>` para **almacenar los elementos**.



Decisión de diseño

El **primer nodo** de la estructura enlazada **representa** la **cima de la pila**.

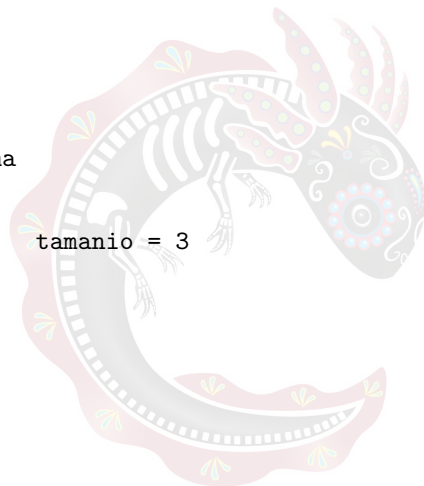
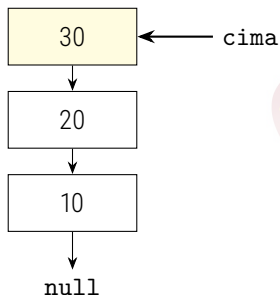
Diagrama de clases



Representación interna

`PilaEnlazada<E>` requiere únicamente:

- Una **referencia** `cima`.
- Un **contador** `tamano`.



Invariantes

- Si `tamano == 0`, entonces `cima == null`.
- Si `tamano > 0`, entonces `cima != null`.

Constructor

Una pila **recién creada** no contiene elementos:

- 1: $cima \leftarrow null$
- 2: $tamano \leftarrow 0$

cima
↓
null

tamano = 0

estaVacia() = true



Método `apilar(elemento)`

Para **insertar** un elemento:

Require: elemento

- 1: $nuevo \leftarrow \text{Nodo}(\text{elemento})$
- 2: $nuevo.siguiente \leftarrow cima$
- 3: $cima \leftarrow nuevo$
- 4: $tamano \leftarrow tamano + 1$



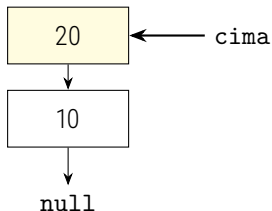
¡Importante!

El **nuevo nodo** se inserta **siempre** al **inicio de la estructura enlazada**.

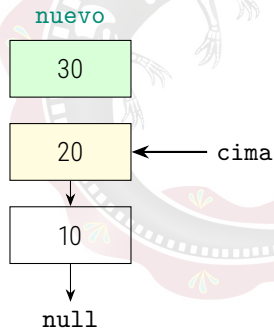
$$\text{apilar}() = O(1)$$

apilar(30): crear el nodo

Estado inicial:

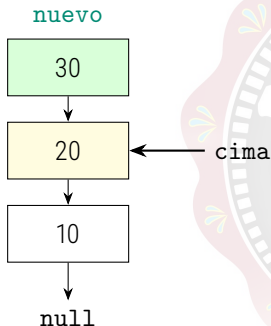


```
Nodo<E> nuevo = new Nodo<>(30);
```



apilar(30): crear el enlace

```
nuevo.setSiguiente(cima)
```

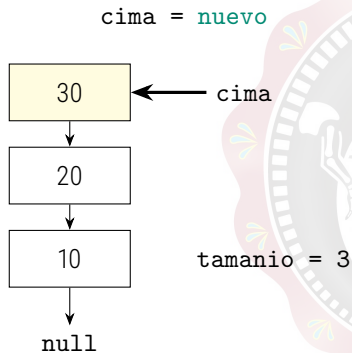


Resultado parcial

El nuevo nodo **ya está conectado** con la pila, pero `cima` **todavía** referencia al nodo 20.

apilar(30): actualizar cima

Finalmente:

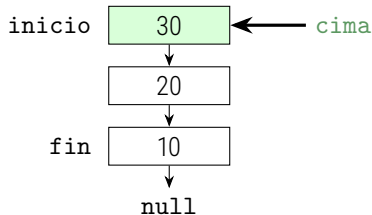


30 → 20 → 10 → null

¿Por qué la cima está al inicio?

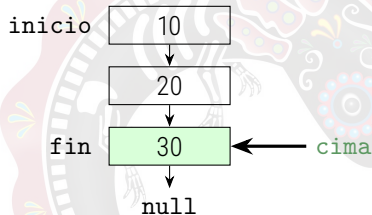
Consideremos dos alternativas:

Cima al inicio



$\text{apilar} = O(1)$
 $\text{desapilar} = O(1)$

Cima al final



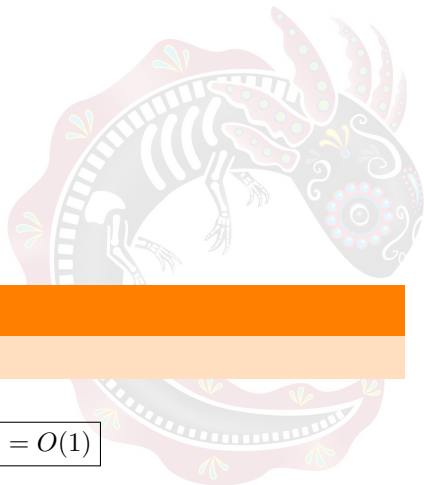
$\text{desapilar} = O(n)$

¡Importante!

Representar la **cima** mediante el **primer nodo** hace **eficientes** las **operaciones fundamentales** de la pila.

Método desapilar()

- 1: verificar que la pila no esté vacía
- 2: *eliminado* $\leftarrow$ *cima*
- 3: *cima* $\leftarrow$ *cima.siguiete*
- 4: *tamano* $\leftarrow$ *tamano* - 1
- 5: **return** *eliminado.elemento*



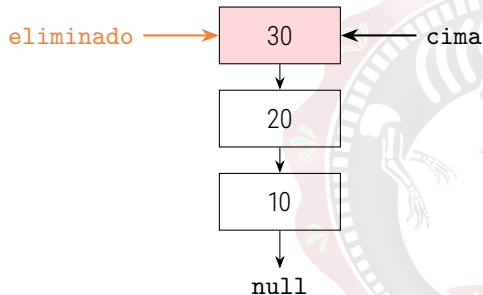
¡Importante!

No es necesario **recorrer** los nodos.

$$\text{desapilar()} = O(1)$$

desapilar(): conservar la cima

Estado inicial:

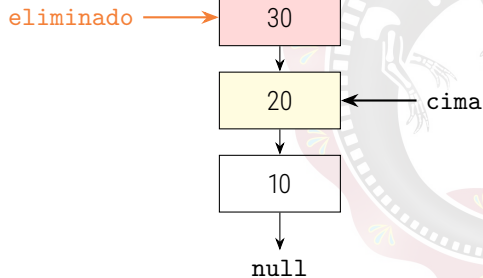


`eliminado = cima`

Esto permite **conservar** el **elemento** que deberá **devolverse**.

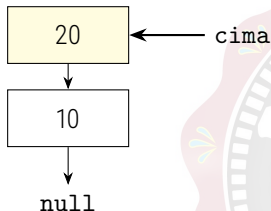
desapilar(): actualizar la cima

```
cima = cima.getSiguiente()
```



desapilar(): actualizar la cima

Finalmente:



`desapilar() = 30`

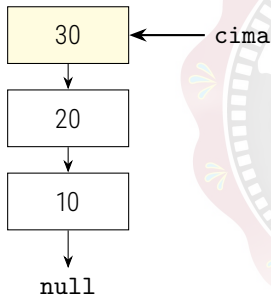
La pila queda:

`20 → 10 → null`

Método `cima()`

La operación `cima()` **consulta** el **elemento superior** **sin modificar la pila**.

- 1: verificar que la pila no esté vacía
- 2: **return** `cima.elemento`



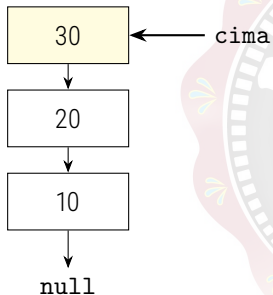
`cima() = 30`

$O(1)$



cima() **vs.** desapilar()

Partimos de:

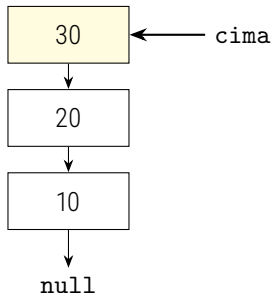


cima() **VS.** desapilar()

cima()

cima() = 30

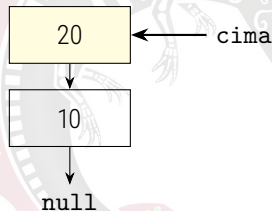
La estructura **permanece:**



desapilar()

desapilar() = 30

La estructura **cambia:**



¡Importante!

Ambos **consultan la cima**, pero **solo desapilar()** **modifica la estructura.**

Pila vacía

¿Qué ocurre si ejecutamos?

```
desapilar()    o    cima()
```

cuando:

```
cima == null
```

- 1: **if** la pila está vacía **then**
- 2: lanzar `IllegalStateException`
- 3: **end if**

¡Importante!

Las **operaciones** `desapilar()` y `cima()` requieren que exista **al menos un elemento**.

Método `verificarNoVacía()`

Para evitar duplicar la validación:

- 1: **if** *tamaño* = 0 **then**
- 2: lanzar `IllegalStateException`
- 3: **end if**

Este método se utiliza desde:

`desapilar()`

`cima()`

Encapsulamiento

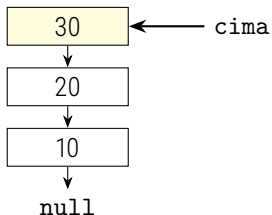
La **validación** forma parte de los **detalles internos** de `PilaEnlazada<E>`, por lo que el método puede declararse como `private`.



Métodos `tamaño()` y `estaVacia()`

`tamaño()`

return *tamaño*



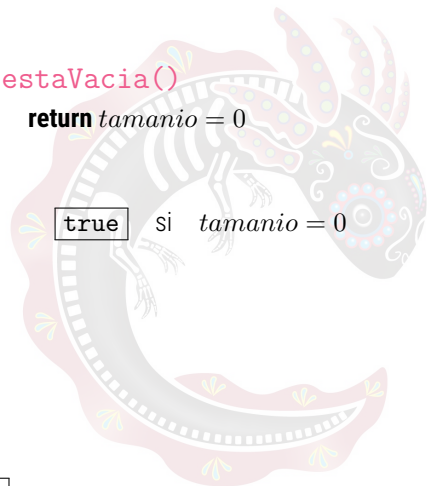
`tamaño() = 3`

`estaVacia()`

return *tamaño* = 0

`true` si *tamaño* = 0

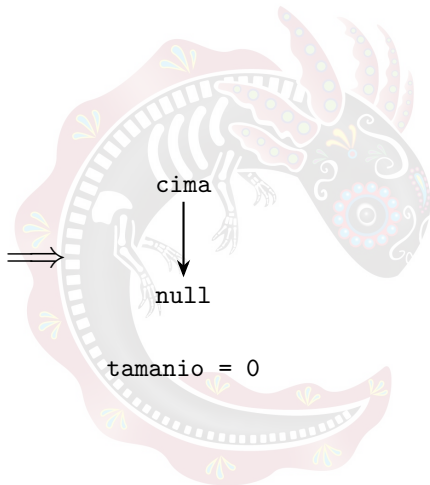
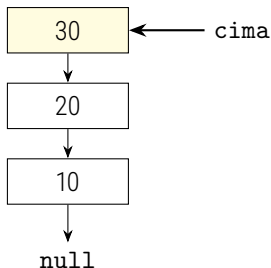
$O(1)$



Método limpiar()

Para eliminar todos los elementos:

- 1: $cima \leftarrow null$
- 2: $tamano \leftarrow 0$



Java

Los nodos **dejan de ser accesibles** desde la pila y **pueden** ser recuperados posteriormente por el **recolector de basura**.

Ejemplo de operaciones

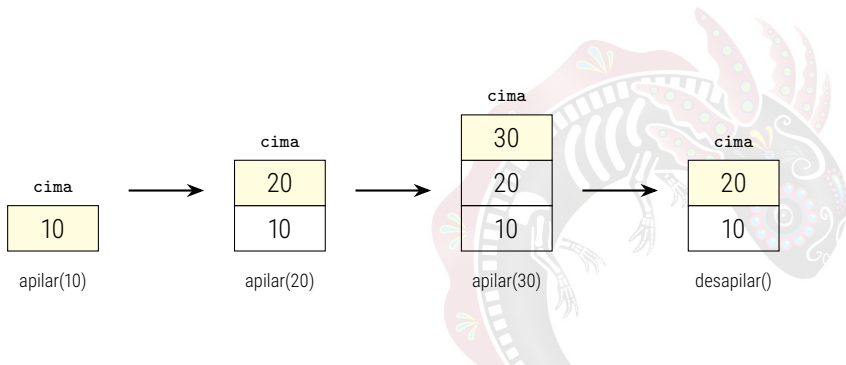
Consideremos la siguiente secuencia:

Operación	Pila	Resultado
apilar(10)	[10]	-
apilar(20)	[20, 10]	-
apilar(30)	[30, 20, 10]	-
cima()	[30, 20, 10]	30
desapilar()	[20, 10]	30
desapilar()	[10]	20

orden de entrada: 10, 20, 30

orden de salida: 30, 20, 10

Secuencia de operaciones



¡Importante!

Todas las **inserciones y eliminaciones** se realizan en el **mismo extremo** de la estructura.

Complejidad temporal

Operación	Complejidad
apilar(elemento)	$O(1)$
desapilar()	$O(1)$
cima()	$O(1)$
tamano()	$O(1)$
estaVacía()	$O(1)$
limpiar()	$O(1)$

¡Importante!

Las operaciones del TDA **no requieren recorrer** la estructura enlazada.

Lista enlazada vs. pila enlazada

Característica	ListaEnlazada<E>	PilaEnlazada<E>
Componente básico	Nodo<E>	Nodo<E>
Acceso por posición	Sí	No
Inserción	Diferentes posiciones	Solo en la cima
Eliminación	Diferentes posiciones	Solo en la cima
Búsqueda	Sí	No forma parte del TDA
Principio de acceso	Por posición	LIFO

Idea importante

Ambas estructuras **pueden** utilizar los mismos nodos, pero **implementan TDA diferentes**.

¿Por qué Pila no extiende Lista?

Una **pila** **debe permitir**:

apilar desapilar cima

Una **lista** permite **operaciones adicionales** como:

agregar(indice) obtener(indice) eliminar(indice)

¡Importante!

Si **Pila heredara** todas las **operaciones** de **Lista**, sería posible **modificar posiciones** arbitrarias, **rompiendo la abstracción LIFO**.

Diseño

La implementación puede **reutilizar conceptos o estructuras** enlazadas **sin exponer** operaciones que **no pertenecen** al TDA Pila.

Invariantes de PilaEnlazada

Después de cualquier **operación válida**:

1 Si la **pila está vacía**:

`tamano == 0` $\iff$ `cima == null`

2 Si **existen elementos**:

`cima != null`

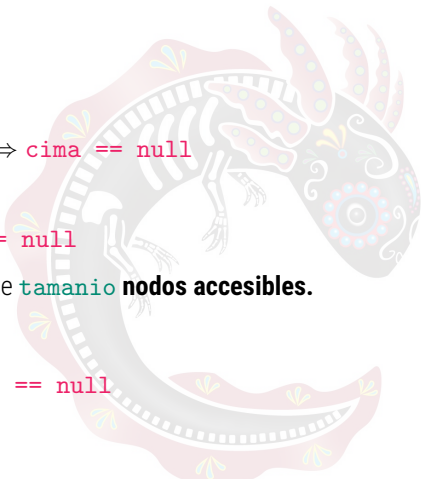
3 Partiendo de `cima`, existen exactamente `tamano` **nodos accesibles**.

4 El **último nodo** de la cadena cumple:

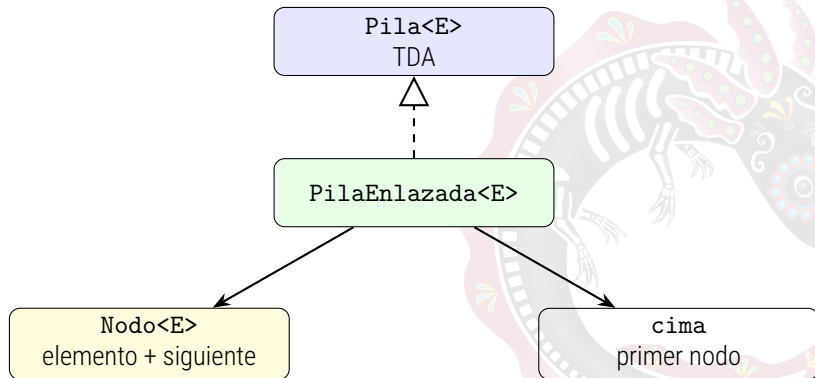
`siguiente == null`

Uso

Estas **propiedades** pueden utilizarse para **comprobar** que `apilar()` y `desapilar()` mantienen un **estado válido** de la estructura.



Implementación de PilaEnlazada<E>



Idea fundamental

PilaEnlazada<E> implementa el comportamiento **LIFO** haciendo que la **cima** corresponda al **primer nodo** de una estructura simplemente enlazada.

Fin de la presentación

¿Preguntas o comentarios?

